

Reinforcement Learning

Class 8

Integrating learning and planning

Mostly based on D. Silver (UCL) lecture 8 from 2015; chapter 8 from Sutton & Barto's book

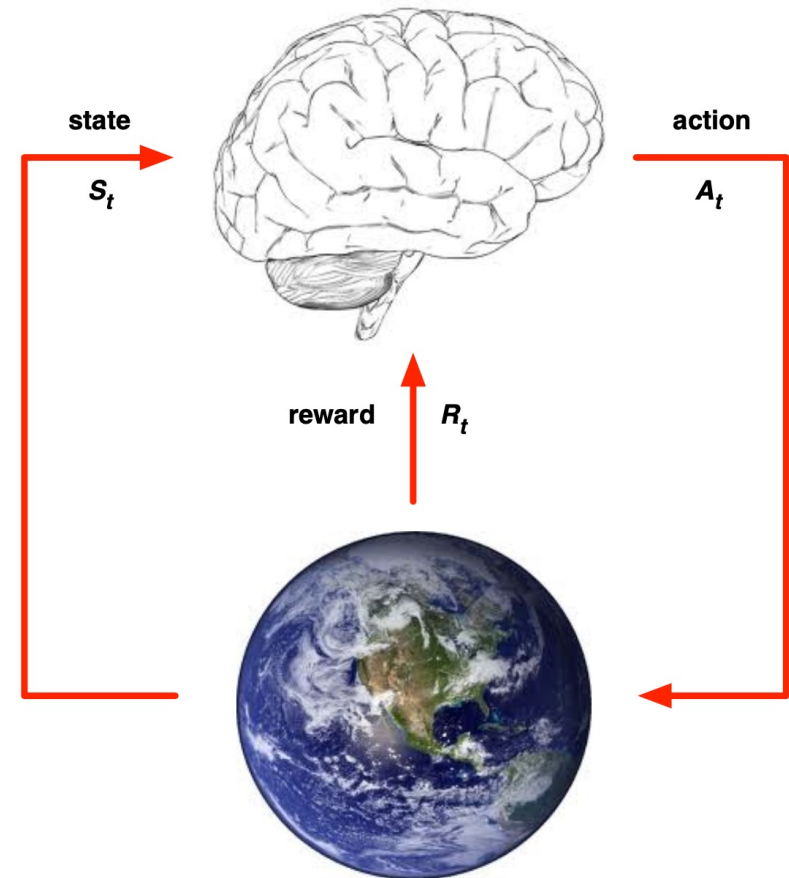
Context

- We have covered the core RL methods in the last classes, including the framework of MDPs, how to reach optimal policies when knowing MDPs and how to handle unknown MDPs to reach optimal policies with TD, MC and Q-learning
- Then, we looked at **approximation methods** that allow to address more realistic scenarios with many states and continuous cases, focusing on value (or state-value) functions, and then using **policy gradient**, where we will address **directly the optimization of policies**
- In this class, we will cover learning **models** of the environment dynamics from experience and then using them with planning algorithms (integrate **learning + planning**) in an unified view of model-free and model-based methods

Model based vs model free

- **Model-Free RL**

- **No model** of the world (environment)
- **Learn** value functions or policies directly from experience



Model based vs model free

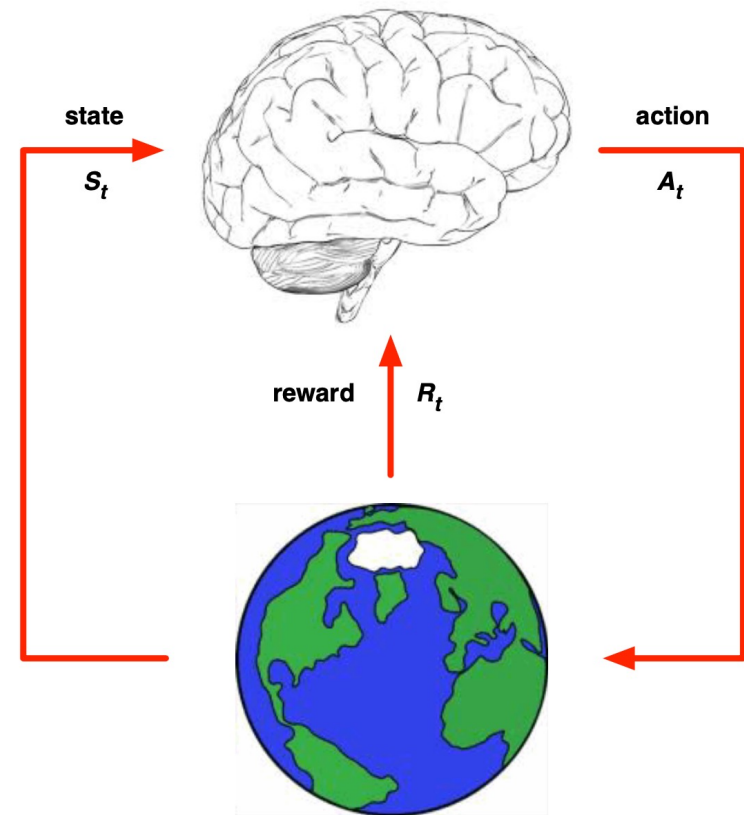
- **Model-Free RL**

- No model of the world (environment)
- **Learn** value functions or policies directly from experience

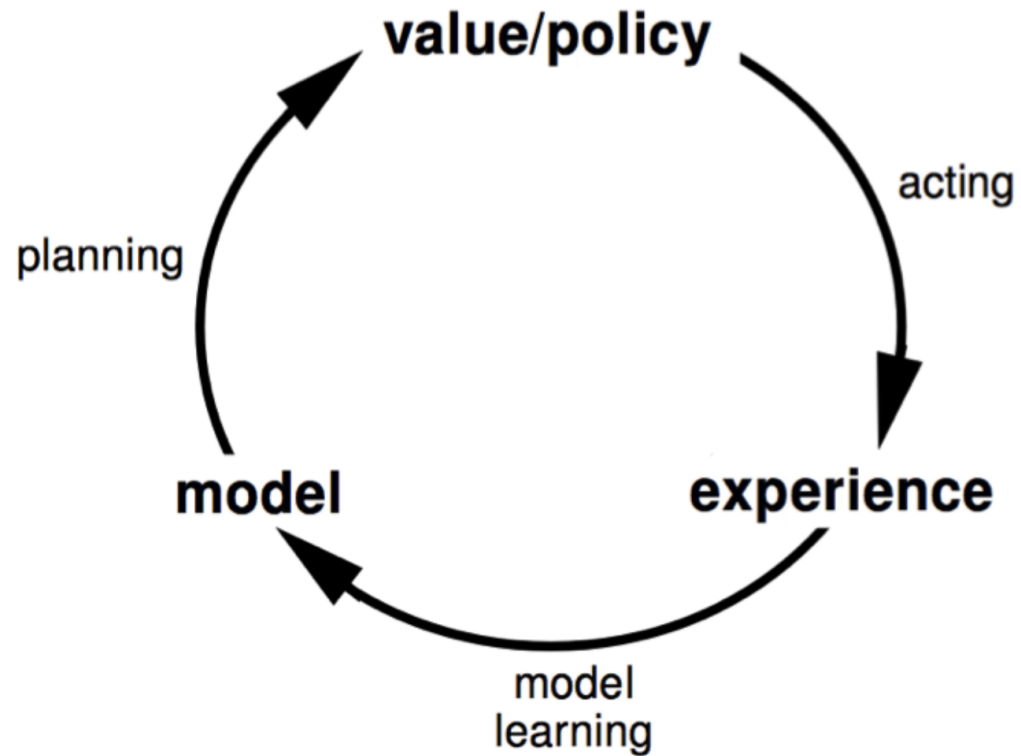
- **Model-based RL**

- Learn a model of the world from experience
- **Plan** value function and/or policy from model

A **model** will be defined as anything that an agent can use to predict how the environment will react to its actions (next **state** and issued **reward**)



Model-based reinforcement learning



Advantages of model-based RL

- **Advantages**

- Can efficiently learn model by supervised learning methods
- Can reason about model uncertainty

- **Disadvantages**

- First learn a model, then construct a value function => two sources of approximation error

What is a model ?

- A **model** M is a representation of an MDP $\langle S, A, P, R \rangle$ parametrized by η
- We will assume state space S and action space A are known
- So, a model M represents state transitions and rewards

$$S_{t+1} \sim \mathcal{P}_{\eta}(S_{t+1} \mid S_t, A_t)$$

$$R_{t+1} = \mathcal{R}_{\eta}(R_{t+1} \mid S_t, A_t)$$

- Models can be used to **simulate** the environment generating complete episodes from a starting state and policy (simulated experience)
- Typically, we assume conditional independence between state

transitions and rewards

$$\mathbb{P}[S_{t+1}, R_{t+1} \mid S_t, A_t] = \mathbb{P}[S_{t+1} \mid S_t, A_t] \mathbb{P}[R_{t+1} \mid S_t, A_t]$$

Types of models

- **Distribution models**

- Produce a description of all possibilities and their probabilities
- It is the model assumed in Dynamic Programming and to fully define the MDP

- **Sample models**

- Produce one possibility samples from the probabilities
- Easier to reach in practice

- Distribution models are stronger and will be considered over the next slides; they can be used to produce samples, and therefore are more general

Model learning

- Goal: estimate model M_η from experience $\{S_1, A_1, R_2, \dots, S_T\}$
- This is a **supervised learning problem** with training set:

$$S_1, A_1 \rightarrow R_2, S_2$$

$$S_2, A_2 \rightarrow R_3, S_3$$

$$\vdots$$

$$S_{T-1}, A_{T-1} \rightarrow R_T, S_T$$

- Learning $s, a \rightarrow r$ is a **regression** problem
- Learning $s, a \rightarrow s'$ (distribution model) is a **density estimation** problem
- Pick loss function, e.g. mean-squared error, KL divergence, ...
- Find parameters η that minimise empirical loss

Examples of models

- **Table Lookup Model** – simplest case !
- Linear Expectation Model – linear model over feature space
- Linear Gaussian Model
- Gaussian Process Model
- Deep Belief Network Model
- ...

Table lookup model

- Model is an explicit representation of MDP
- Count visits $N(s,a)$ to each state-action pair

$$\hat{\mathcal{P}}_{s,s'}^a = \frac{1}{N(s,a)} \sum_{t=1}^T \mathbf{1}(S_t, A_t, S_{t+1} = s, a, s')$$

$$\hat{\mathcal{R}}_s^a = \frac{1}{N(s,a)} \sum_{t=1}^T \mathbf{1}(S_t, A_t = s, a) R_t$$

- Alternatively, at each time-step t , record experience tuple $\langle S_t, A_t, R_{t+1}, S_{t+1} \rangle$
- To sample model, randomly pick tuple matching $\langle s, a, \cdot, \cdot \rangle$

Example – AB example

- Two states, no discounting; 8 episodes of experience

A, 0, B, 0

B, 1

B, 1

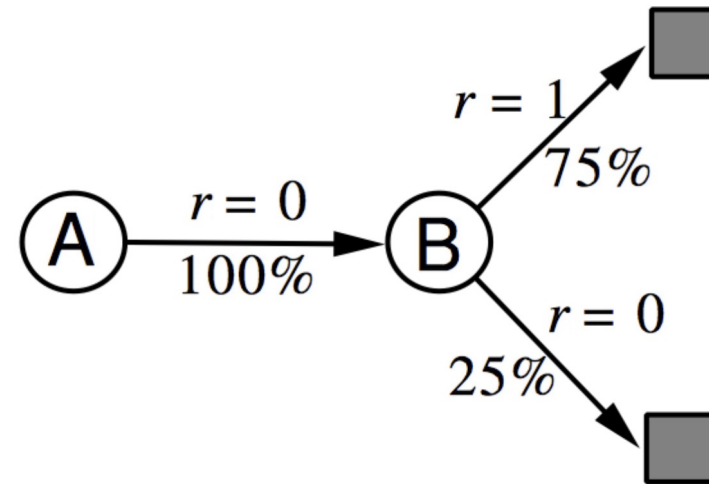
B, 1

B, 1

B, 1

B, 1

B, 0



- We have constructed a table lookup model from the experience

Planning with a model

- Given a model $M_\eta = \langle P_\eta, R_\eta \rangle$
- **Planning** means any computational process that takes a model M_η as input and produces/ improves a policy
- Solve the MDP $\langle S, A, P_\eta, R_\eta \rangle$

using any (or your favourite) planning algorithm

- Value iteration (from dynamic programming)
- Policy iteration (from dynamic programming)
- Tree search
- ...

Sample-based planning

- A simple but powerful approach to planning
- Use the model only to generate samples
- **Sample** experience from model

$$S_{t+1} \sim \mathcal{P}_\eta(S_{t+1} \mid S_t, A_t)$$

$$R_{t+1} = \mathcal{R}_\eta(R_{t+1} \mid S_t, A_t)$$

- Apply **model-free RL to samples**, e.g.:
 - Monte-Carlo control
 - Sarsa
 - Q-learning
- Sample-based planning methods are often more efficient

Pseudo-code for random-sample tabular Q-planning

Random-sample one-step tabular Q-planning

Loop forever:

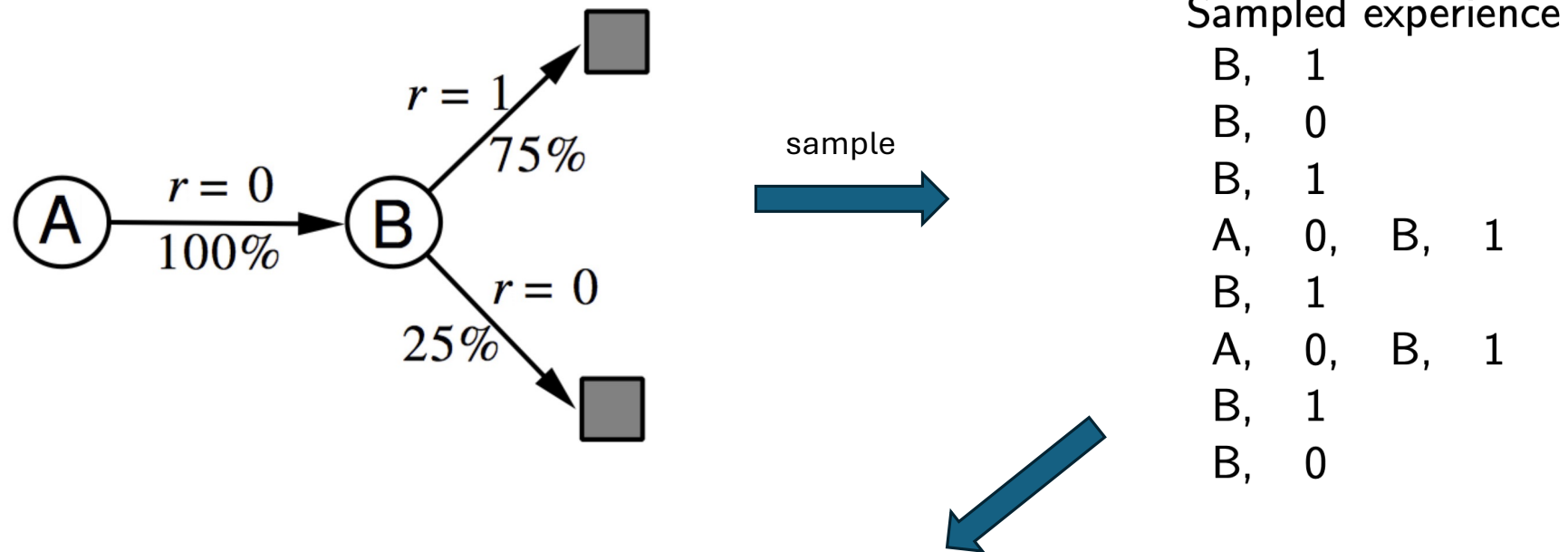
1. Select a state, $S \in \mathcal{S}$, and an action, $A \in \mathcal{A}(S)$, at random
2. Send S, A to a sample model, and obtain
a sample next reward, R , and a sample next state, S'
3. Apply one-step tabular Q-learning to S, A, R, S' :
$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

Pseudo-code

...

Example – AB example

- Construct a table-lookup model from real experience (as before)



- Apply model-free RL to sampled experience
e.g. Apply Monte-Carlo learning: $V(A) = 1, V(B) = 0.75$

Planning with an Inaccurate Model

- Given an imperfect model $\langle \mathcal{P}_\eta, \mathcal{R}_\eta \rangle \neq \langle \mathcal{P}, \mathcal{R} \rangle$
- Performance of model-based RL is limited to optimal policy for approximate MDP $\langle S, A, P_\eta, R_\eta \rangle$
i.e. **Model-based RL is only as good as the estimated model**
- When the model is inaccurate, planning process will compute a suboptimal policy
- Solution 1: when model is wrong, use model-free RL
- Solution 2: reason explicitly about model uncertainty

Integrating learning and planning

Real and simulated experience

- We can consider two sources of experience
- **Real experience** - Sampled from environment (true MDP)

$$S' \sim \mathcal{P}_{s,s'}^a$$

$$R = \mathcal{R}_s^a$$

- **Simulated experience** - Sampled from the model being learned (approximate MDP)

$$S' \sim \mathcal{P}_\eta(S' \mid S, A)$$

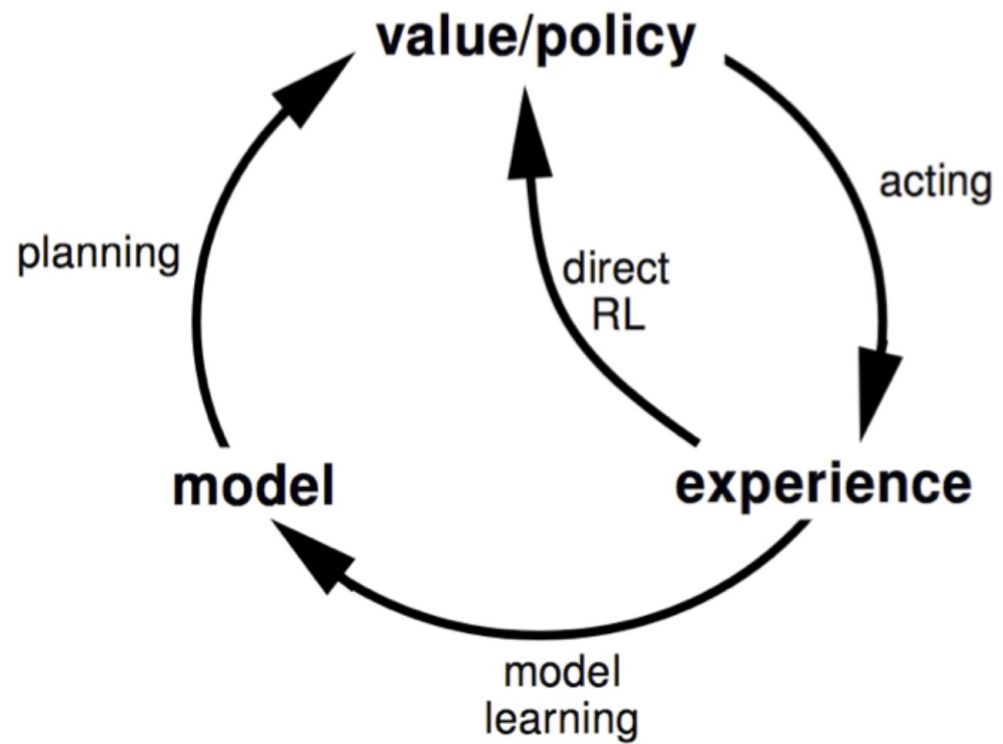
$$R = \mathcal{R}_\eta(R \mid S, A)$$

- We may want information gained from real experience to be used to improve the model and for planning (value function/ policy)

Integrating learning and planning

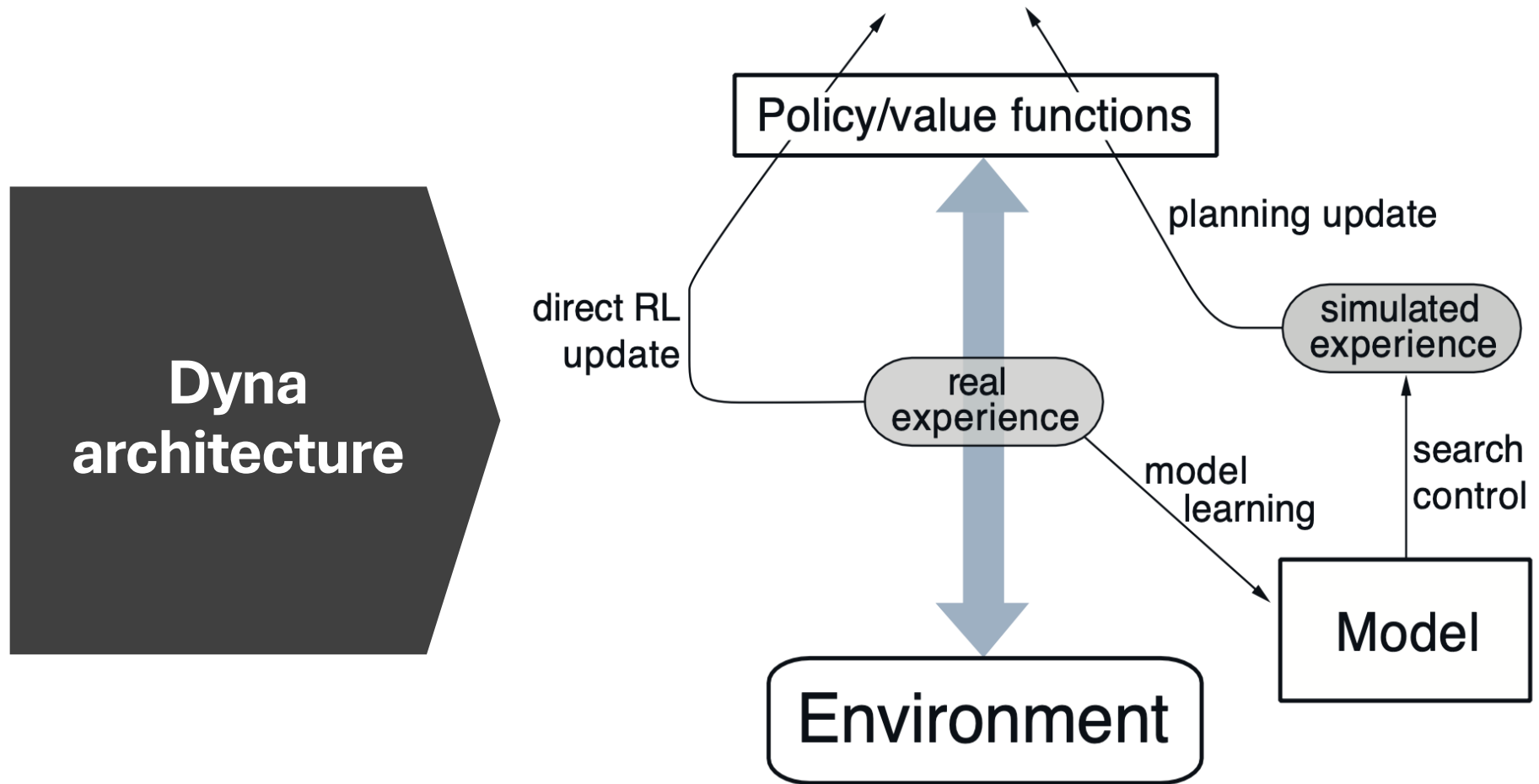
- Model-Free RL
 - No model of the world (environment)
 - Learn value functions or policies directly from experience
- Model-based RL
 - Learn a model of the world from experience
 - Plan value function and/or policy from model
- **Dyna**
 - Learn a **model** from real experience
 - Learn and plan **value function** (and/or policy) from real and simulated experience

Dyna approach



Dyna approach

- Combines direct RL with indirect RL, trying to make the best of both, minimizing their limitations
- **Direct RL**
 - experience is used directly to improve estimates of value function and/ or policies
 - simpler to run and not affected by biases in the model
- **Indirect RL**
 - The improvement of the value functions/ policies is done through the learning of a model of the environment followed by planning based on simulated experience
 - Can make fuller use of the limited available data and may be able to achieve better results



Learning and planning are deeply integrated - they share almost all the same machinery, differing only in the source of their experience

Pseudo-code for tabular Dyna-Q

Tabular Dyna-Q

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

(a) $S \leftarrow$ current (nonterminal) state

(b) $A \leftarrow \varepsilon$ -greedy(S, Q)

(c) Take action A ; observe resultant reward, R , and state, S'

(d) $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

(e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)

(f) Loop repeat n times:

$S \leftarrow$ random previously observed state

$A \leftarrow$ random action previously taken in S

$R, S' \leftarrow Model(S, A)$

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

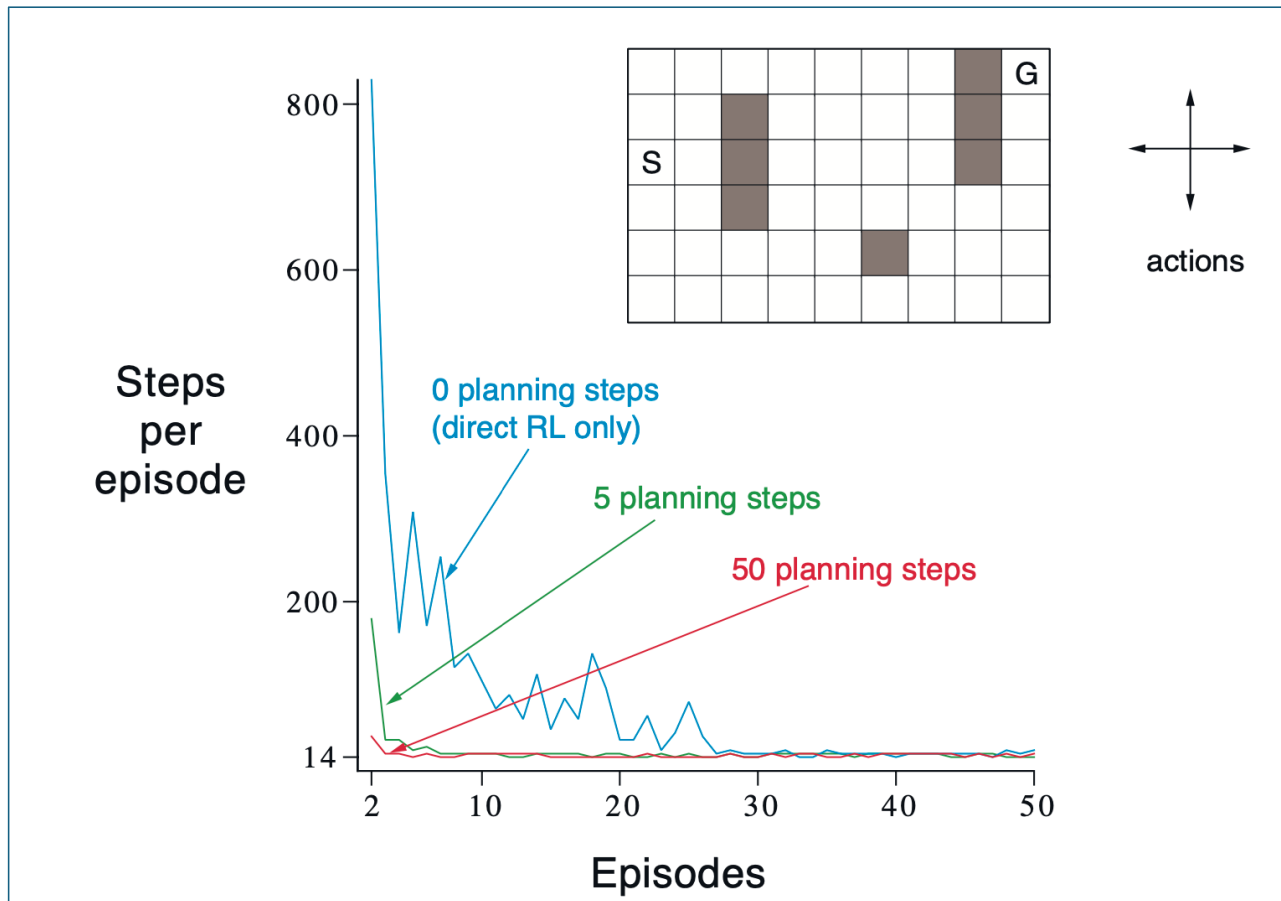
Step (d) – Q-learning from real experience (direct RL)

Step (e) updates the model with the new case (model learning)

Step (f) Q-learning from experience from the model (planning)

Pseudo-code

Example – Dyna Q on a simple maize



Different scenarios using a different n – number of planning steps using the model

Example from Sutton & Barto, ch. 8

Example – Dyna Q on a simple maize

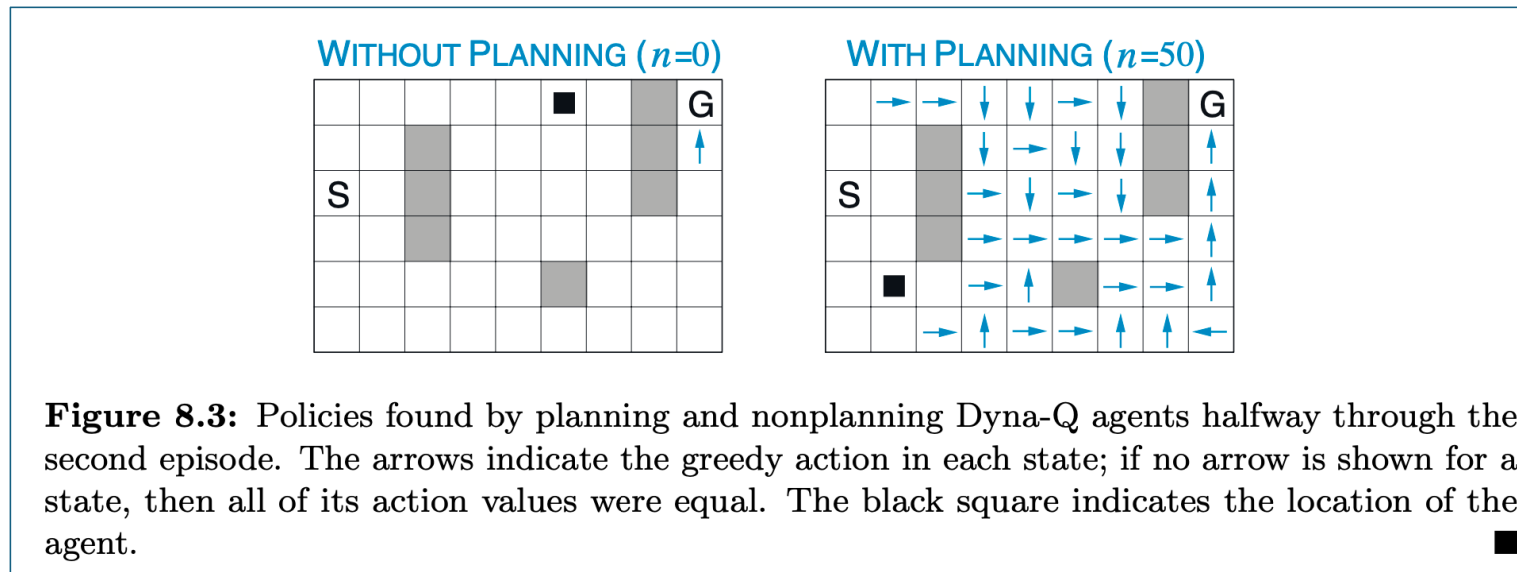
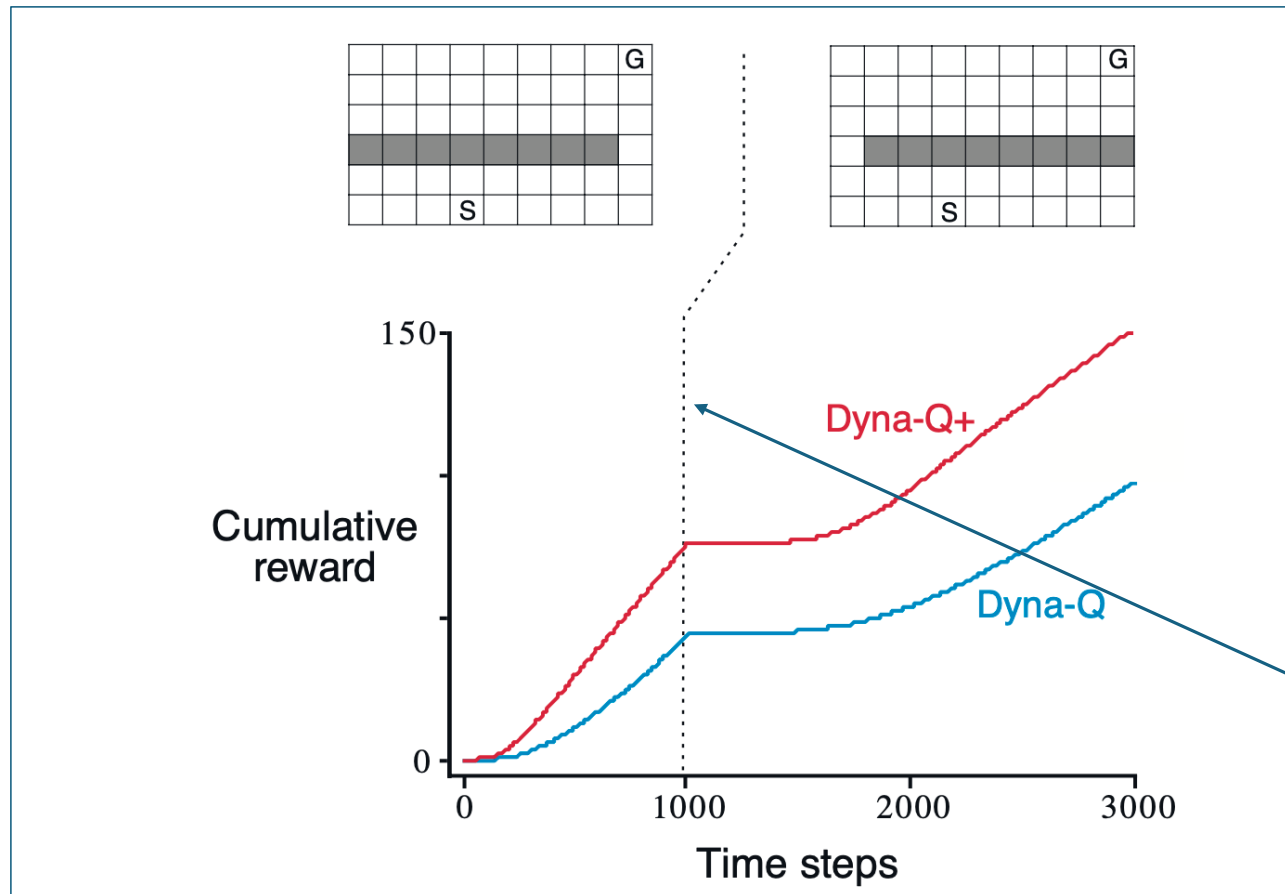


Figure 8.3: Policies found by planning and nonplanning Dyna-Q agents halfway through the second episode. The arrows indicate the greedy action in each state; if no arrow is shown for a state, then all of its action values were equal. The black square indicates the location of the agent.

Without planning ($n = 0$), each episode adds only one additional step to the policy, and so only one step (the last) has been learned so far. With planning, again only one step is learned during the first episode, but here during the second episode an extensive policy has been developed that by the end of the episode will reach almost back to the start state.

Example – Dyna Q with innacurate model

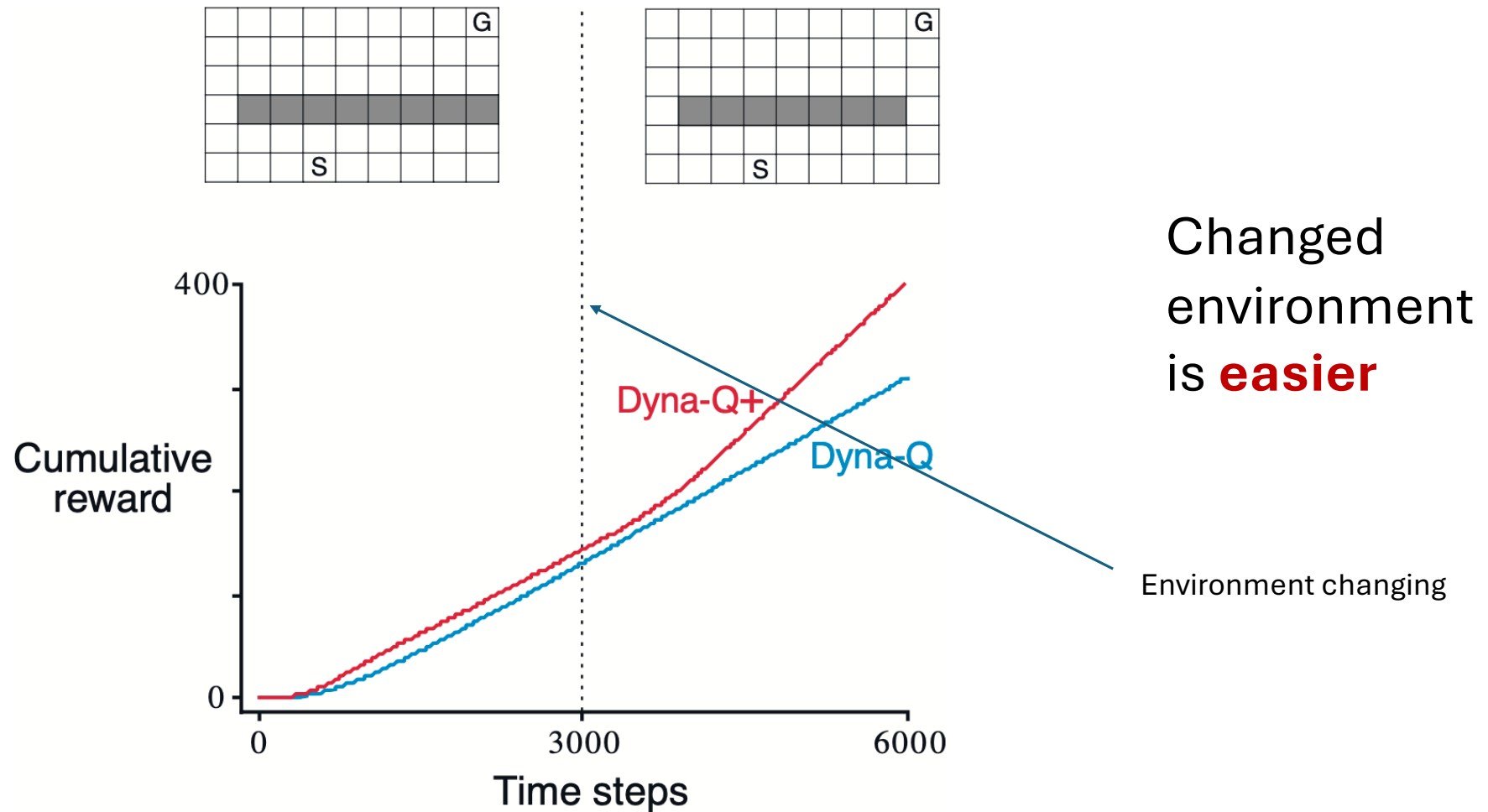


Changed environment is **harder**

Environment changing

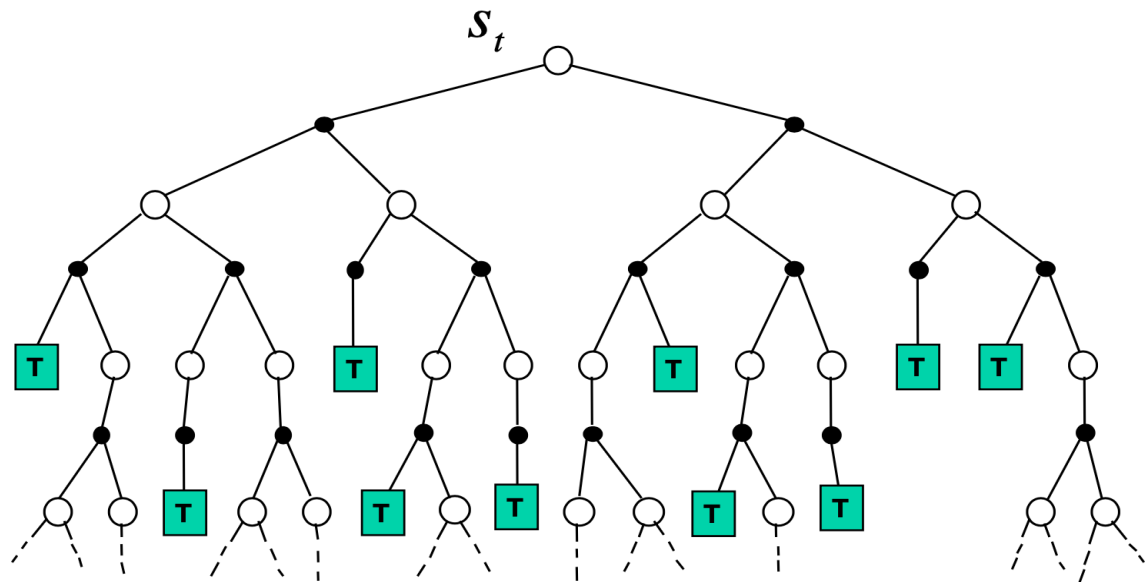
Dyna-Q+ encourages further exploration to the Dyna-Q

Example – Dyna Q with innacurate model



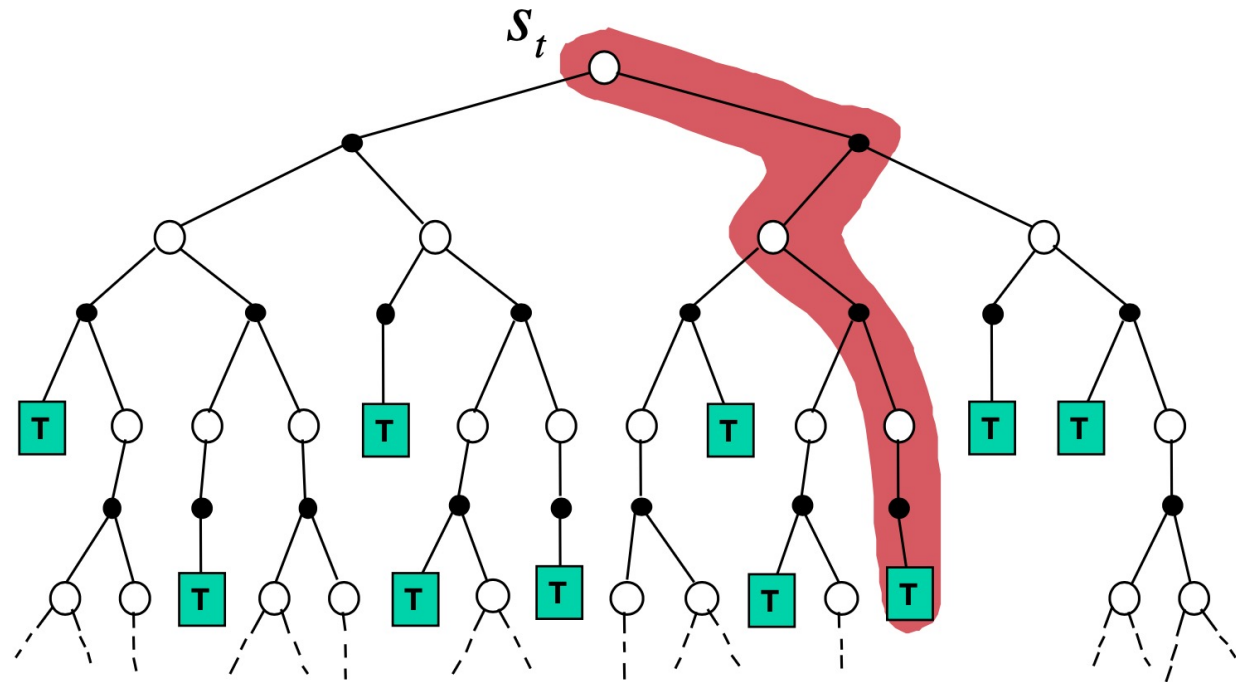
Forward search

- **Forward search** algorithms select the best action by lookahead
- They build a **search tree** with the **current state** s_t at the root
- Using a **model** of the MDP to **look ahead**
- No need to solve whole MDP, just the **sub-MDP starting from now (current state)**



Simulation-based search

- **Forward search** paradigm using **sample-based planning**
- Simulate **episodes of experience from now** with the **model**
- Apply **model-free RL** to simulated episodes



Simulation-based search

- Simulate multiple episodes (K times) of experience from now with the **model**

$$\{s_t^k, A_t^k, R_{t+1}^k, \dots, S_T^k\}_{k=1}^K \sim \mathcal{M}_\nu$$

- Apply model-free RL to simulated episodes
 - Monte-Carlo control → **Monte-Carlo search**
 - Sarsa → **TD search**

Simple Monte-Carlo search

- Given a model M_ν and select a **simulation policy** π (aka rollout policy)
- For each action $a \in \mathcal{A}$
 - Simulate K episodes from current (real) state s_t

$$\{\mathbf{s}_t, \mathbf{a}, R_{t+1}^k, S_{t+1}^k, A_{t+1}^k, \dots, S_T^k\}_{k=1}^K \sim \mathcal{M}_\nu, \pi$$

- Evaluate actions by mean return (Monte-Carlo evaluation)

$$Q(\mathbf{s}_t, \mathbf{a}) = \frac{1}{K} \sum_{k=1}^K G_t \xrightarrow{P} q_\pi(s_t, a)$$

- Select current (real) action with maximum value

$$a_t = \operatorname{argmax}_{a \in \mathcal{A}} Q(s_t, a)$$

Monte-Carlo Tree Search - evaluation

- Given a model M_ν
- Simulate K episodes from current state s_t using **current simulation policy** π

$$\{\mathbf{s}_t, A_t^k, R_{t+1}^k, S_{t+1}^k, \dots, S_T^k\}_{k=1}^K \sim \mathcal{M}_\nu, \pi$$

- Build a **search tree containing visited states and actions**
- Evaluate states $Q(s,a)$ by mean return of episodes from s,a

$$Q(\mathbf{s}, \mathbf{a}) = \frac{1}{N(s, a)} \sum_{k=1}^K \sum_{u=t}^T \mathbf{1}(S_u, A_u = s, a) G_u \xrightarrow{P} q_\pi(s, a)$$

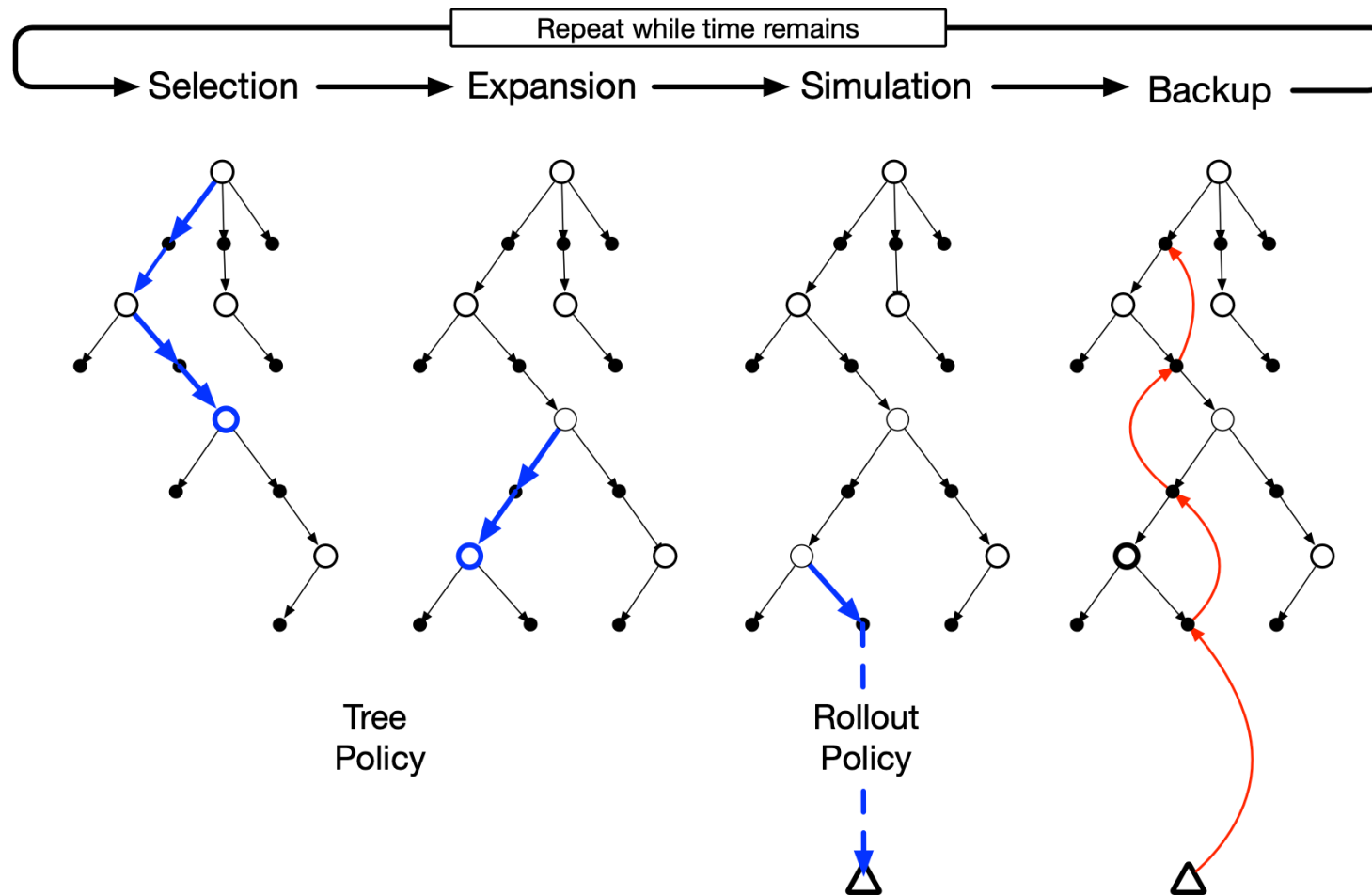
- After **search is finished**, select current (real) action with maximum value in search tree

$$a_t = \operatorname{argmax}_{a \in \mathcal{A}} Q(s_t, a)$$

Monte-Carlo Tree Search - simulation

- In MCTS, the **simulation (tree) policy π improves**
- Each simulation consists of two phases
 - **Tree policy (improves)**: pick actions to maximise $Q(S,A)$ – in-tree
 - **Default rollout policy (fixed)**: pick actions randomly – out-of-tree (new)
- Repeat (each simulation)
 - Evaluate states $Q(S,A)$ by Monte-Carlo evaluation
 - Improve tree policy, e.g. by ϵ -greedy(Q)
- This is **Monte-Carlo control** but now applied to simulated experience
- Converges on the optimal search tree, $Q(S,A) \rightarrow q_*(S,A)$
- Q values may be approximated by function approximation (AlphaGo)

Monte Carlo Tree Search



Set of steps repeated for each new state reached,

Executes as many iterations as possible to build the tree, before taking the selected action

Relevant parts of the tree may be used in next states

Monte Carlo Tree Search - steps

1. **Selection** – starting at the root node follow the tree policy based on the Q values at edges to traverse the tree to a leaf node
2. **Expansion** – on some iterations, expand the tree from a selected leaf by adding one or more child nodes (using default policy)
3. **Simulation** – from the selected node (or one newly add node), simulate **complete episode** (until final state) with actions first from tree policy and then rollout policy
4. **Backup** – return from the episode used to update or initialize the action values of the traversde edges (from the tree policy only)

These steps are repeated until no more time/ resources are left, and finally an action from the root node (current state) is selected.

In the next state, MCTS is run again typically starting with a tree that may containing descendants of the new node from the tree previously built. The remaining of the tree is discarded.

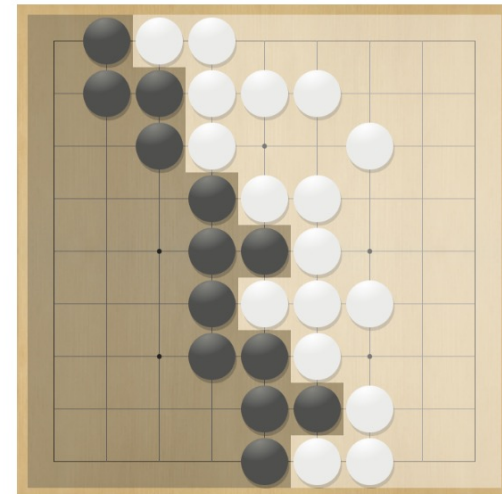
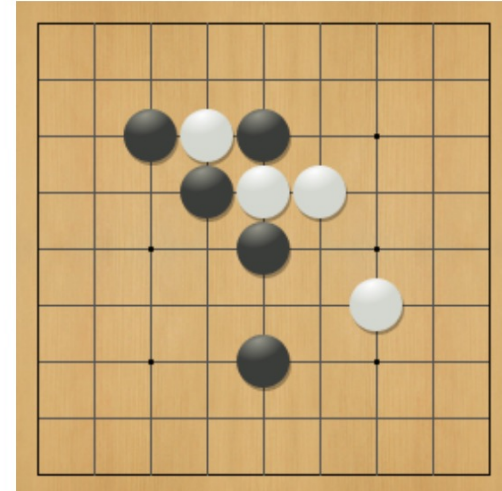
Example – Game of Go

- The ancient oriental game of Go is 2500 years old
- Considered to be the **hardest** classic board game
- Considered a grand challenge task for AI
- Traditional game-tree search has failed in Go, which has a much larger space than chess



Example – Game of Go

- Usually played on 19x19, also 13x13 or 9x9 board
- Simple rules, complex strategy
- Black and white place down stones alternately
- Surrounded stones are captured and removed
- The player with more territory wins the game



Example – Game of Go – position evaluation

- How good is a position s ?
- Reward function (undiscounted):

$$R_t = 0 \text{ for all non-terminal steps } t < T$$

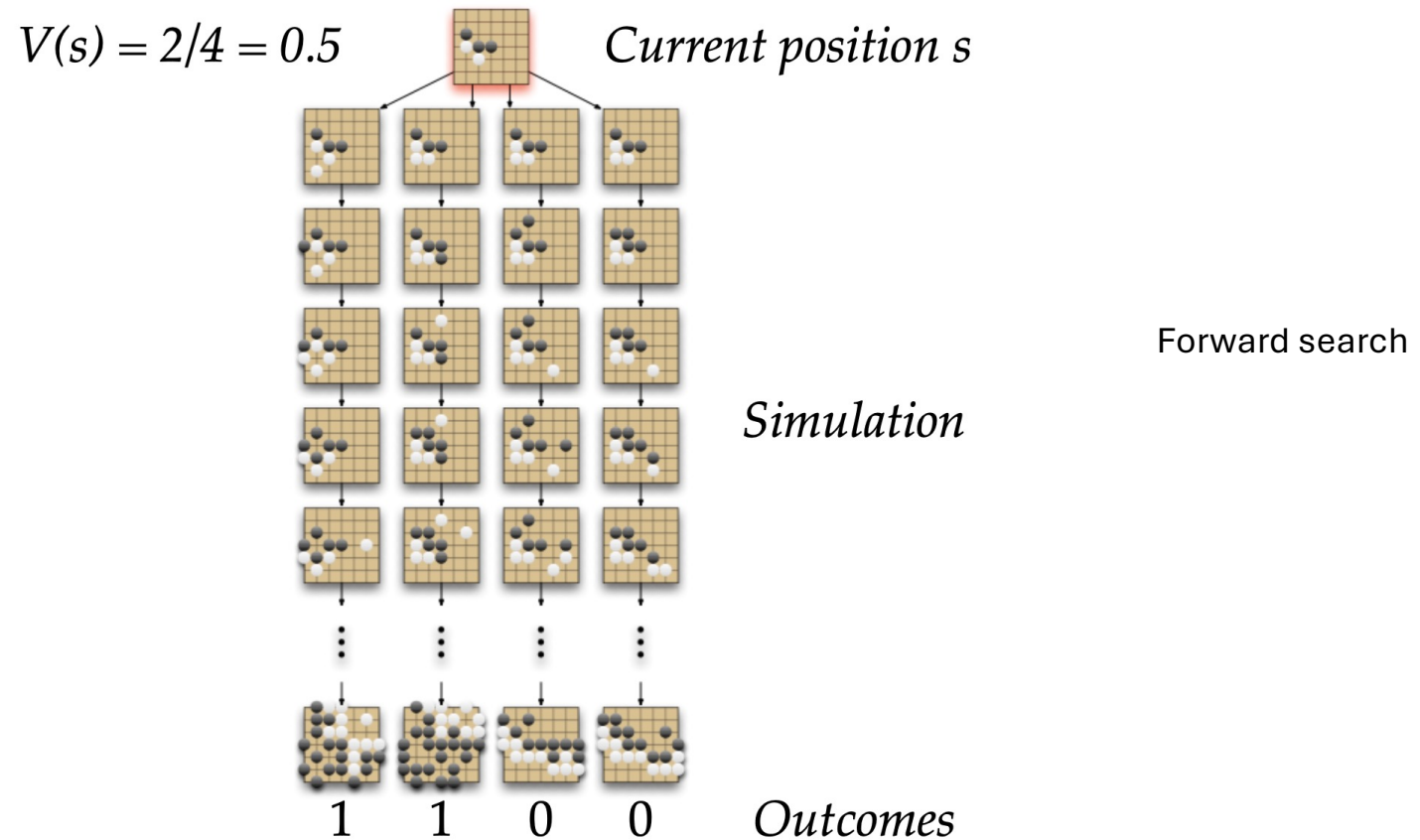
$$R_T = \begin{cases} 1 & \text{if Black wins} \\ 0 & \text{if White wins} \end{cases}$$

- Policy $\pi = \langle \pi_B, \pi_W \rangle$ selects moves for both players
- Value function (how good is position s):

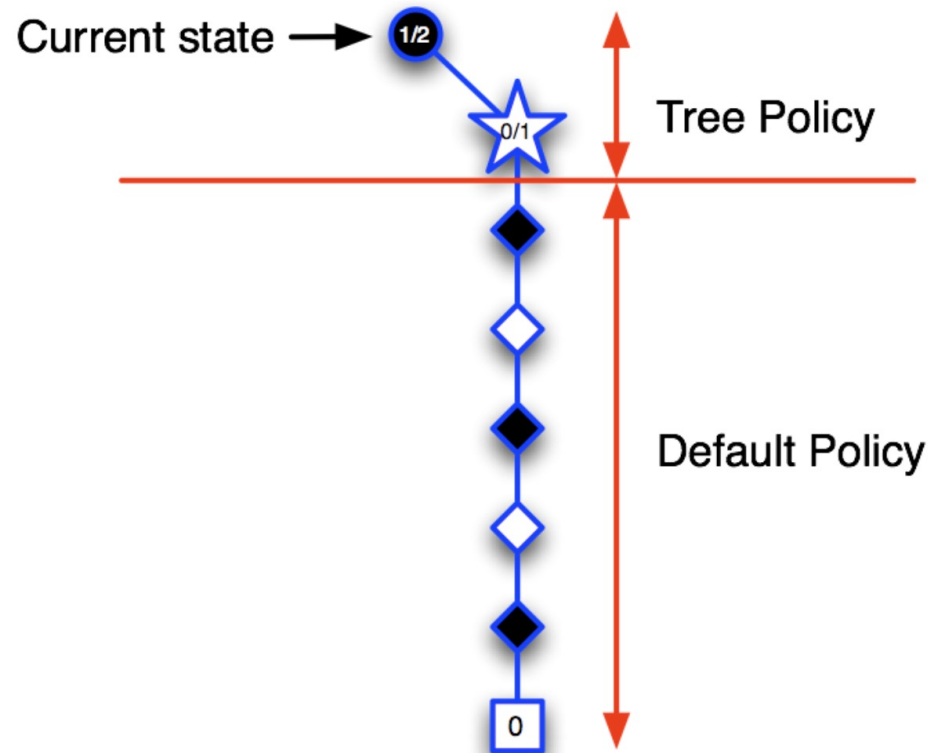
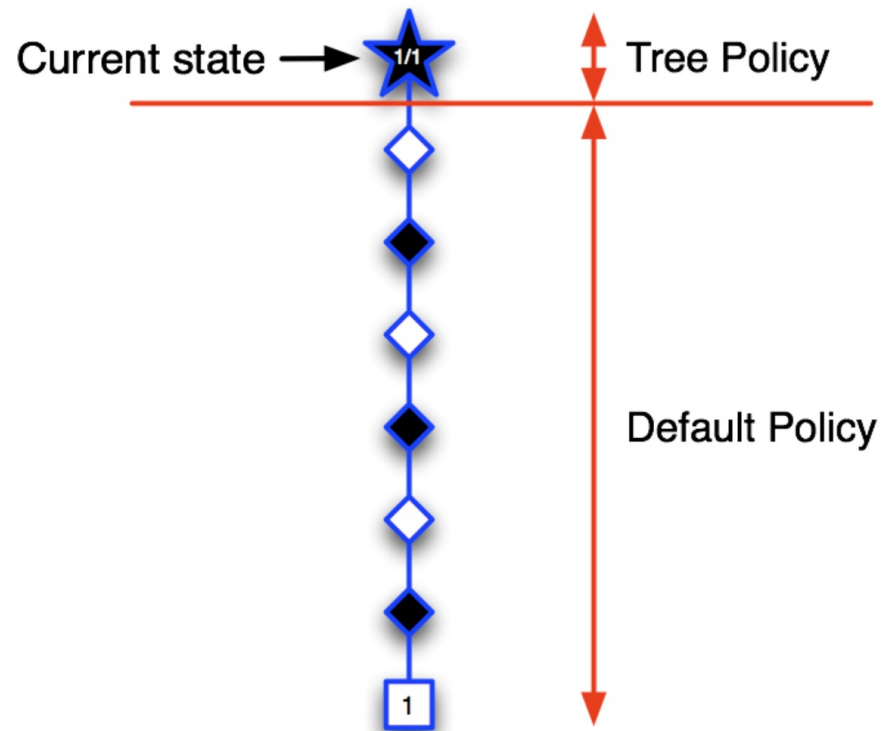
$$v_\pi(s) = \mathbb{E}_\pi [R_T \mid S = s] = \mathbb{P} [\text{Black wins} \mid S = s]$$

$$v_*(s) = \max_{\pi_B} \min_{\pi_W} v_\pi(s)$$

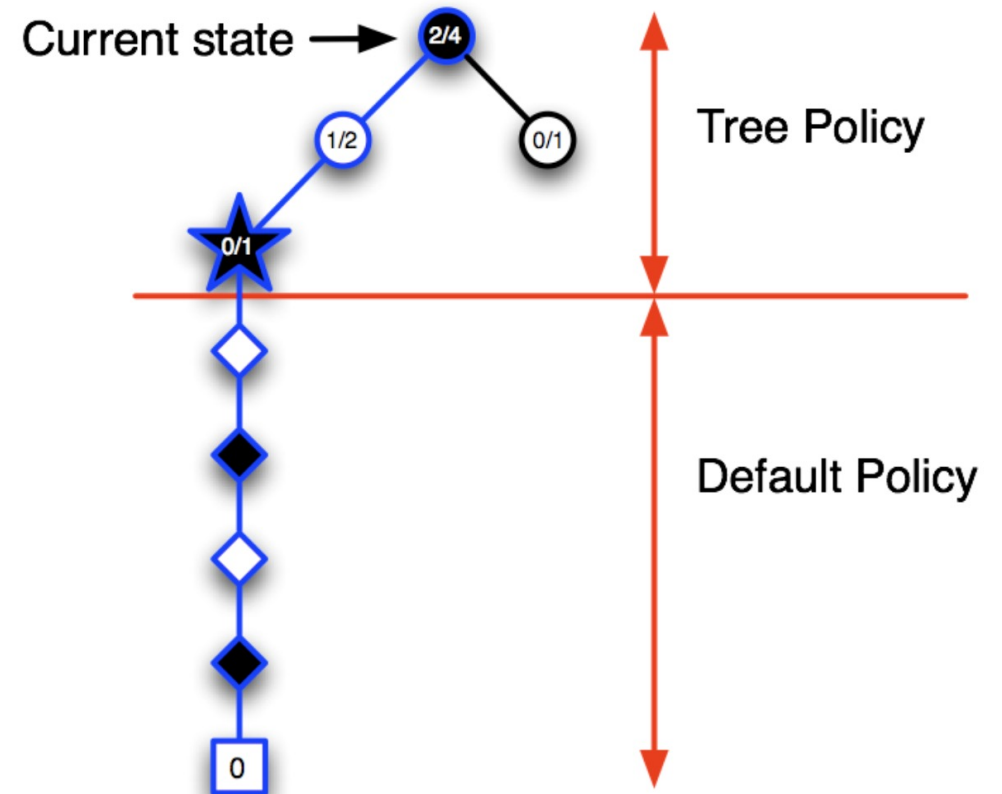
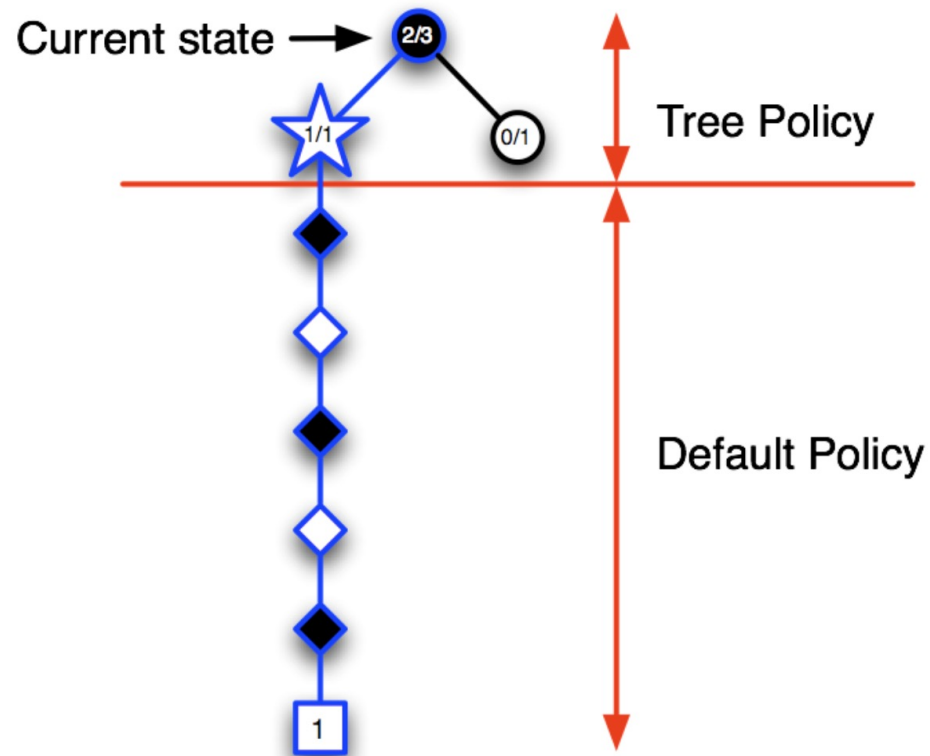
Example – Game of Go – position evaluation



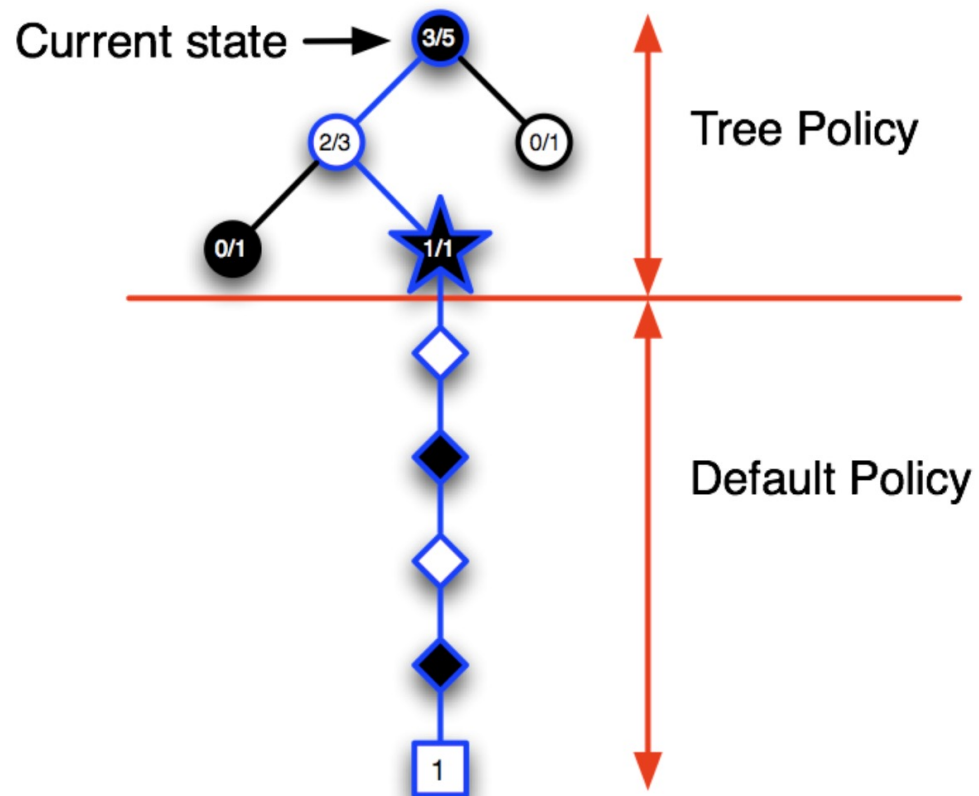
Example – Game of Go – MCTS



Example – Game of Go – MCTS



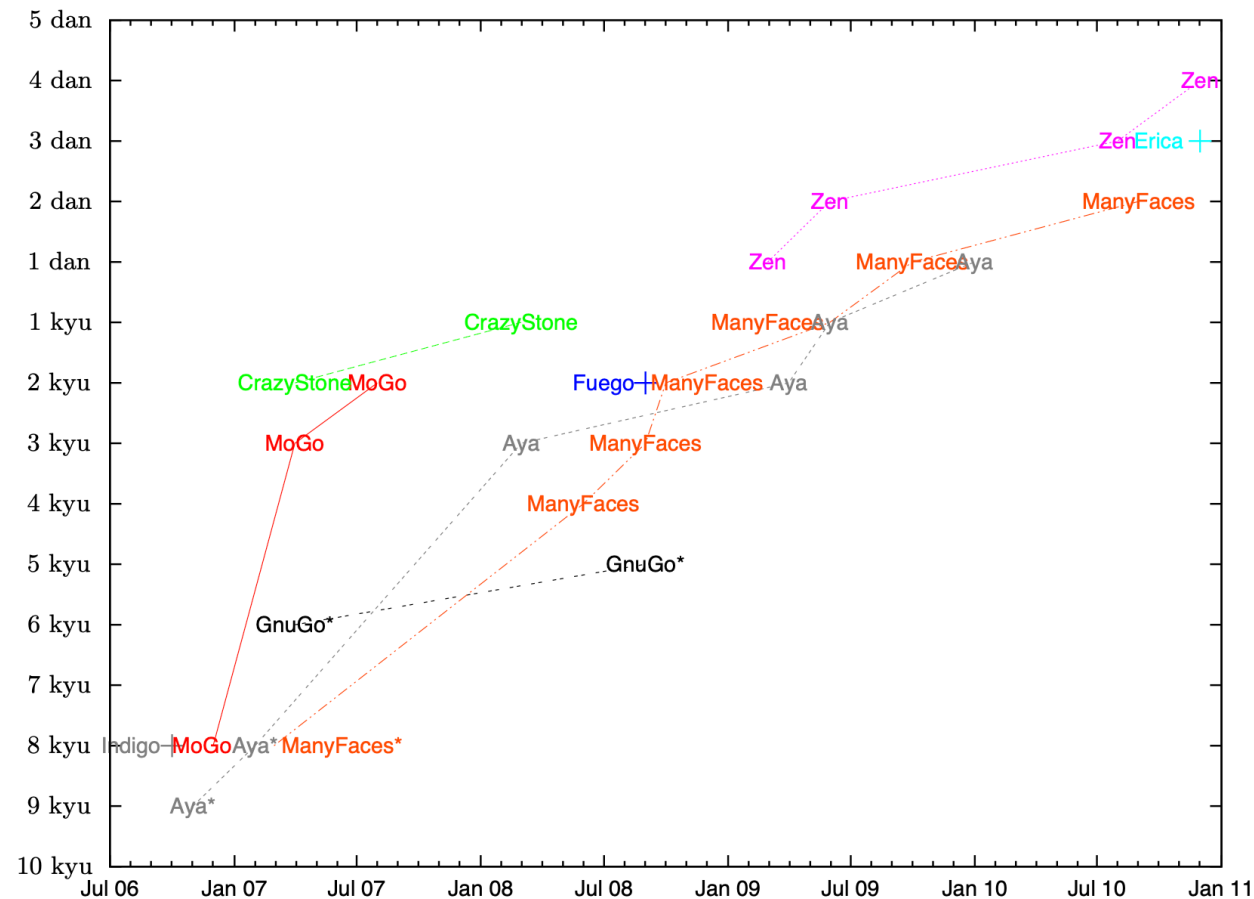
Example – Game of Go – MCTS



Advantages of MC Tree Search

- Highly selective best-first search
- Evaluates states dynamically (unlike e.g. Dynamic Programming)
- Uses sampling to break curse of dimensionality
- Works for “black-box” models (only requires samples)
- Computationally efficient, anytime, parallelisable

Example – Game of Go – MCTS results



TD search

- Simulation-based search
- Using TD instead of MC (bootstrapping)
- MC tree search applies MC control to sub-MDP from now
- TD search applies Sarsa to sub-MDP from now

MC vs TD search

- For **model-free reinforcement learning**, **bootstrapping** is helpful
 - TD learning reduces variance but increases bias
 - TD learning is usually more efficient than MC
 - TD(λ) can be much more efficient than MC
- For **simulation-based** search, **bootstrapping** is also helpful
 - TD search reduces variance but increases bias
 - TD search is usually more efficient than MC search
 - TD(λ) search can be much more efficient than MC search

MC vs TD search

- Simulate episodes from the current (real) state s_t
- Estimate action-value function $Q(s,a)$
- For each step of simulation, update action-values by Sarsa

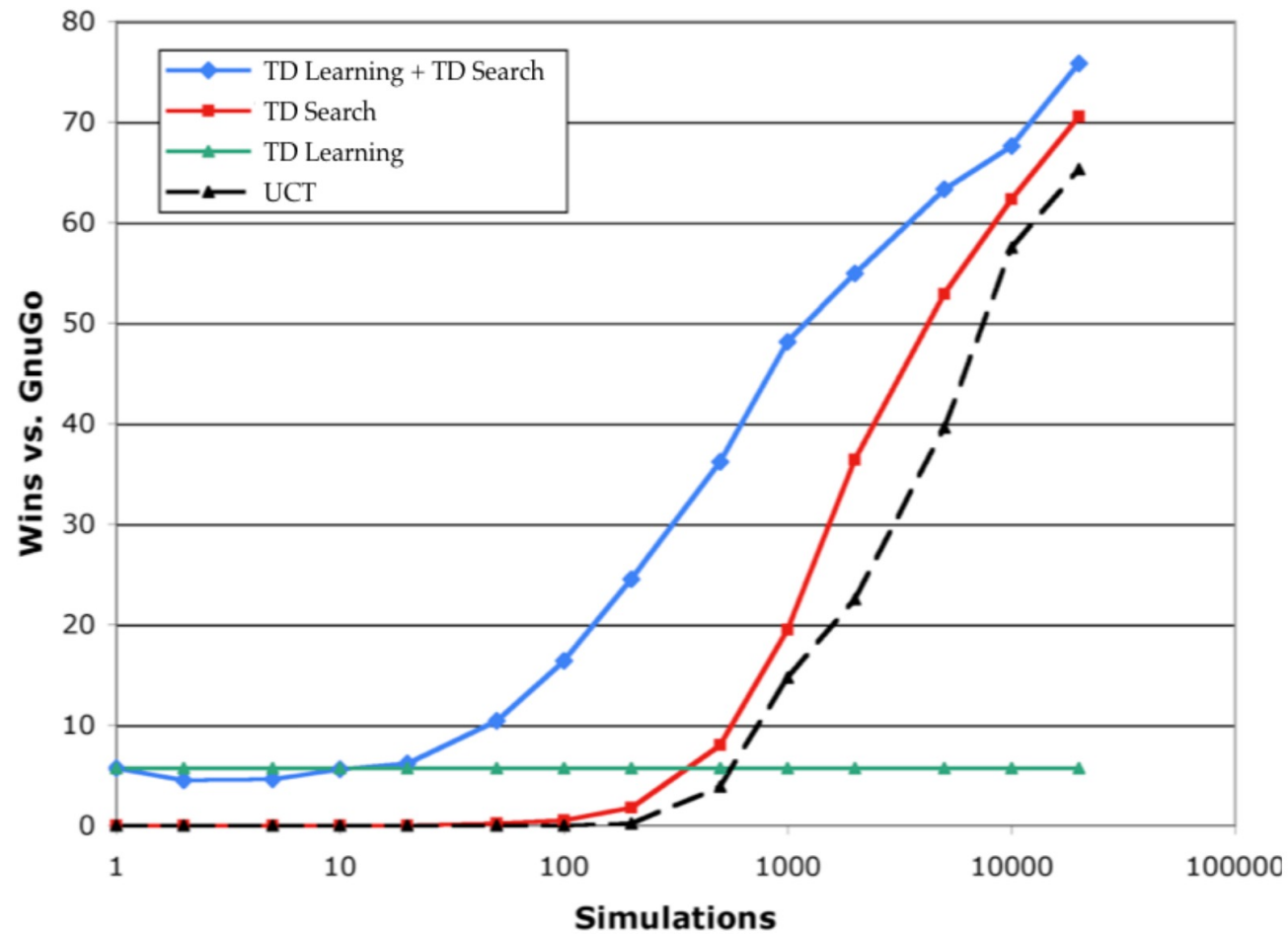
$$\Delta Q(S, A) = \alpha(R + \gamma Q(S', A') - Q(S, A))$$

- Select actions based on action-values $Q(s,a)$
 - e.g. ϵ -greedy
- May also use function approximation for Q

Dyna-2

- In Dyna-2, the agent stores **two sets of feature weights**
 - Long-term memory
 - Short-term (working) memory
- **Long-term memory** is updated from **real** experience
 - using TD learning
 - General domain knowledge that applies to any episode
- **Short-term memory** is updated from **simulated** experience using TD search
 - Specific local knowledge about the current situation
- Overall value function is the sum of long and short-term memories

Example – Game of Go – TD search



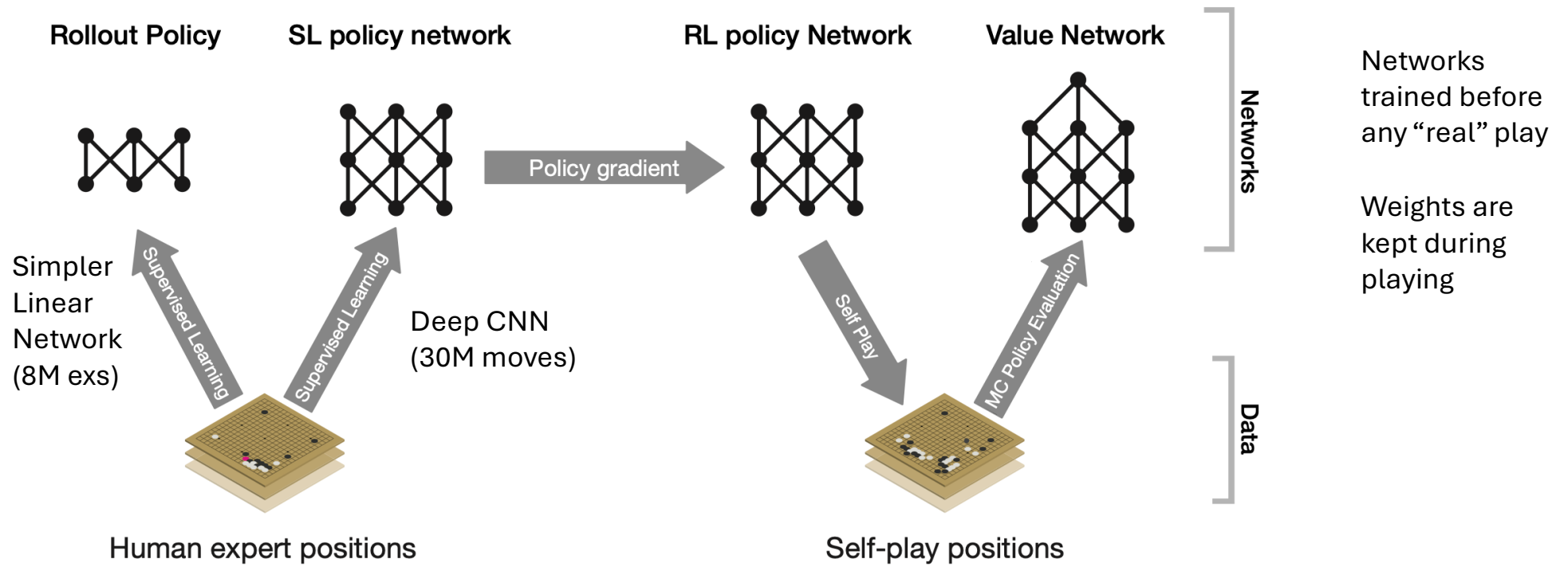
AlphaGo

- A team at **DeepMind** (Silver et al., 2016) developed the program AlphaGo that managed to reach the level of the best players in the world – won 4/5 series against the best human player in 2016
- Combining Deep Convolutional Neural Nets, supervised learning from a database of human expert moves, Monte Carlo tree search, and reinforcement learning
- **AlphaGoZero** (2017) later reached a similar level training only against itself without relying on human moves, thus a more of a pure RL approach
- Both use RL over simulated games of self-play, as well as the ideas of DQNs from DeepMind (Atari games) that used CNNs to approximate optimal value functions

AlphaGo – networks for function approximation

- All networks were **Deep Convolutional networks (CNNs)** with inputs 19x19x48
- **Supervised learning**
 - ***Rollout policy network*** – simple linear network trained from 8 million moves that provided rollout policy
 - ***SL-policy network*** – 13-layer CNN ; input: 19x19x48 (48 features); output: softmax over 19x19 possible positions; trained from a database of nearly 30 million human expert moves; outputs distribution of probabilities over possible actions
- **Reinforcement learning**
 - ***RL-policy network*** – 13-layer CNN; training starts with final weights from SL-policy network; then, policy gradient RL to improve
 - **Value network** – estimates the values of board positions; same structure as *SL-policy network* but single output; trained on a dataset of 30 M moves obtained by **Monte Carlo policy evaluation** from self-play games with moves selected from the RL policy network (opponents were policies from earlier iterations of the algorithm)

AlphaGo - strategy



Function approximation based in CNNs (13 layers, inputs 19x19x48, trained similarly to the DQNs for Atari), both for value functions (Q) and policy gradient

AlphaGo – steps

- Novel version of MCTS – guided by policy and value function learned by RL, using function approximation provided by deep CNNs
- **Asynchronous policy and value MCTS (APV-MCTS)**
 - **Expansion** - choosing an edge according to probabilities supplied by the ***SL-policy network***
 - **Evaluation** of the new node – return of a simulation initiated from it using rollout policy (as in simple MCTS) + output of the value network

$$v(s) = (1 - \eta)v_{\theta}(s) + \eta G$$

- **Action selection** in the end of each step – select the edge (action) most visited

AlphaGo Zero

- Used no human data or guidance beyond rules of the game
- Implements a form of policy iteration – policy iteration + improvement
- AlphaGo Zero used MCTS to select moves throughout self-play RL, whereas AlphaGo used MCTS for play after but not during learning
- AlphaGo Zero only uses one CNN with 19x19x17 image stack as inputs (included position of pieces over last 8 configurations); two outputs – probability of winning from that position + move probabilities
- Does not perform rollout (so does not need rollout policy)
- Output probabilities of the CNN used to direct execution of the MCTS

AlphaGo Zero

a) Self play

The program played many games against itself, one shown here as a sequence of board positions s_i

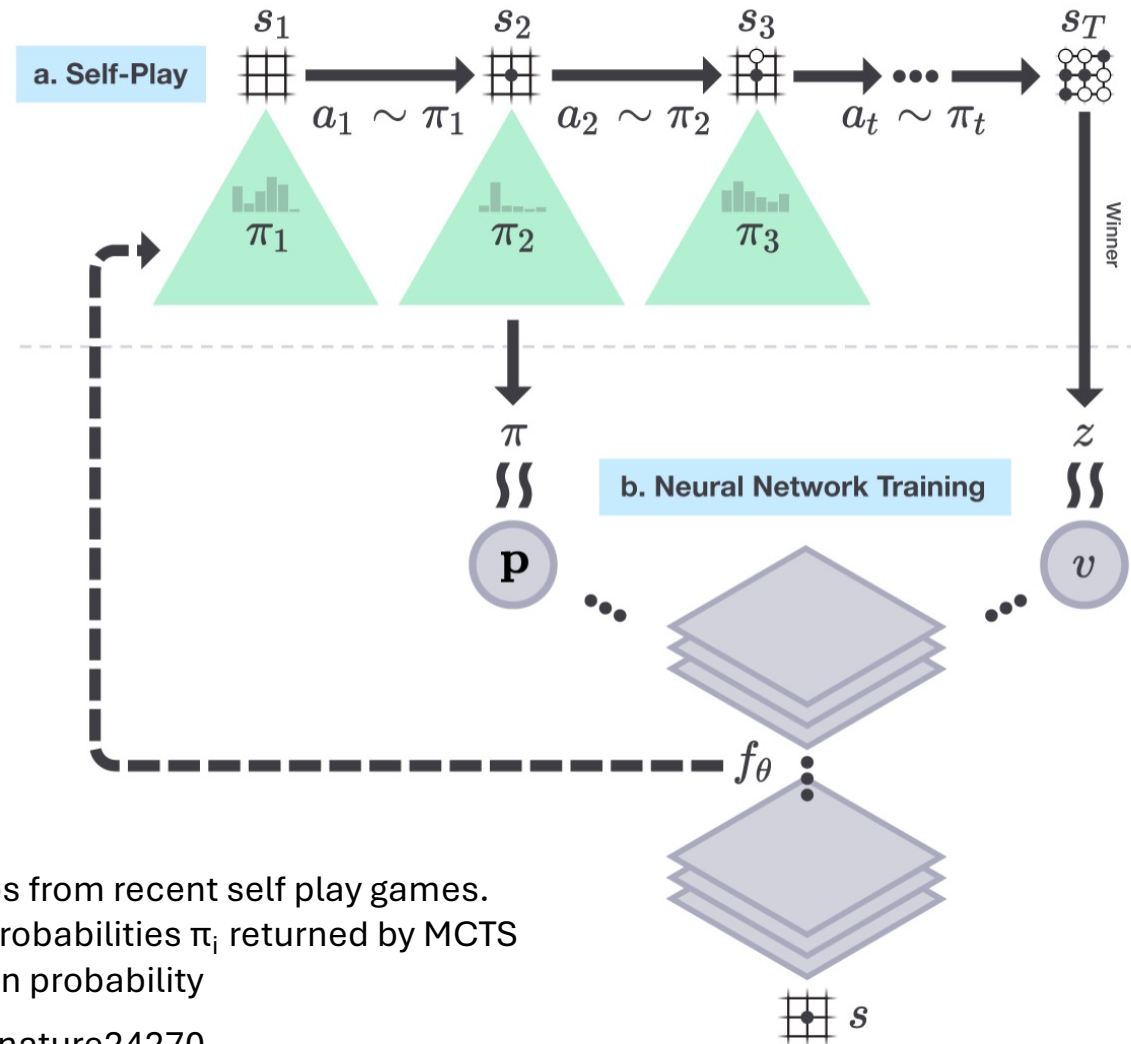
Each move a_i was determined by action probabilities from π_i returned by the MCTS executed from root node s_i and guided by the CNN f_θ

Network inputs were raw representations of board positions (and past positions); outputs are vectors of move probabilities $\mathbf{p}$ that guided MCTS's forward searches + scalar values $\mathbf{v}$ that give the probability of winning

b) CNN training

Training examples were randomly sampled steps from recent self play games. Weights were updated to move $\mathbf{p}$ towards the probabilities π_i returned by MCTS and to include the winners $\mathbf{z}$ in the estimated win probability

More details: <https://www.nature.com/articles/nature24270>



AlphaZero

AlphaZero extends the same core approach into a **general framework** that can learn multiple games, such as chess, shogi, and Go, using the same algorithm with minimal adjustments.

Its significance is that it demonstrates how a unified learning system can master different complex environments.

More details:

<https://www.science.org/doi/10.1126/science.aar6404>

A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play

DAVID SILVER, THOMAS HUBERT, JULIAN SCHRIETTWIESER, IOANNIS ANTONOGLOU, MATTHEW LAJ, ARTHUR GUEZ, MARC LANCTOT, LAURENT SIFRE, DHARSHAN [...] AND DEMIS HASSABIS [+3 authors](#) [Authors Info & Affiliations](#)

SCIENCE • 7 Dec 2018 • Vol 362, Issue 6419 • pp. 1140-1144 • DOI:10.1126/science.aar6404

☐ ☐ ☐ ☐ CHECK

One program to rule them all

Computers can beat humans at increasingly complex games, including chess and Go. However, these programs are typically constructed for a particular game, exploiting its properties, such as the symmetries of the board on which it is played. Silver *et al.* developed a program called AlphaZero, which taught itself to play Go, chess, and shogi (a Japanese version of chess) (see the Editorial, and the Perspective by Campbell). AlphaZero managed to beat state-of-the-art programs specializing in these three games. The ability of AlphaZero to adapt to various game rules is a notable step toward achieving a general game-playing system.

Science, this issue p. [1140](#); see also pp. [1087](#) and [1118](#)

Abstract

The game of chess is the longest-studied domain in the history of artificial intelligence. The strongest programs are based on a combination of sophisticated search techniques, domain-specific adaptations, and handcrafted evaluation functions that have been refined by human experts over several decades. By contrast, the AlphaGo Zero program recently achieved superhuman performance in the game of Go by reinforcement learning from self-play. In this paper, we generalize this approach into a single AlphaZero algorithm that can achieve superhuman performance in many challenging games. Starting from random play and given no domain knowledge except the game rules, AlphaZero convincingly defeated a world champion program in the games of chess and shogi (Japanese chess), as well as Go.